

AgentFlow-Eval Agent 自动化评测工作台

软件著作权登记申请 — 全套材料

著作权人：李凯昕

开发完成日期：2026 年 7 月 14 日

版 本 号：V1.0

文档生成日期：2026 年 7 月

材料一：软件主要功能与技术特点

材料一：软件主要功能与技术特点（申请表专用）

软件全称： AgentFlow-Eval Agent 自动化评测工作台

版本号： V1.0

著作权人 / 开发人： 李凯昕

开发方式： 独立开发

开发完成日期： 2026 年 7 月 14 日

面向场景与需求

随着大语言模型在客服、办公自动化、知识问答等企业场景中的深入应用，业务系统越来越依赖具备多步骤推理与工具调用能力的 Agent（智能体）。传统人工抽检成本高、覆盖面窄，仅对比最终自然语言答案又难以发现“用错工具、多调工具、中间推理偏离”等问题，导致 Agent 质量回归难以工程化落地。AgentFlow-Eval Agent 自动化评测工作台面向企业 AI 应用开发者、算法评测人员与质检人员，提供可重复、可追溯、可评分的自动化评测能力，将业务预期固化为测试用例（TestSuite），通过异步调度执行、完整执行轨迹（Trace）记录与多维指标分（MetricScore）定位问题，形成可运行的 Web 工作台形态，支持本地单机与容器化部署，不强制依赖公网即可完整运行。

核心功能模块拆解

本软件已实现端到端评测闭环，主要功能包括：（1）评测任务（Task）管理：支持创建、分页查询、状态筛选、执行、取消、归档与删除，并可将模型名称、温

度、最大迭代轮次等参数写入任务级 Agent 配置；（2）测试用例（TestSuite）管理：支持在任务下批量维护用例，提供 CSV/JSON 文件上传导入，字段覆盖用户输入、期望输出与期望工具列表；（3）Agent 执行与工具沙箱：基于 ReAct 多轮推理模式调用兼容 OpenAI 协议的大模型接口，内置计算器、检索模拟、时间查询等沙箱工具，具备超时与输出长度控制；（4）执行轨迹（Trace）与可视化：将每一步思考、动作、观察、Token 与耗时结构化落库，前端以 DAG 链路图、步骤日志与评分卡片联动展示；（5）混合评分与报告：先计算规则指标，再可选调用 LLM-as-Judge 精修生成指标分（MetricScore），并支持人工复核覆盖机评分，最终在报告模块聚合展示；（6）企业级横切能力：API Key 鉴权、基于创建者的轻量多租户隔离、审计日志（AuditLog）、接口限流与 WebSocket 任务活动推送。上述功能均已具备可点击、可运行的完整界面与接口形态。

技术架构与特点

系统采用前后端分离的 B/S 架构。表现层使用 TypeScript、React 18、Vite、Ant Design 与 ReactFlow 构建工作台；业务服务层以 Python 3.11+、FastAPI 提供 REST 与 WebSocket，通过 Celery 与 Redis 实现异步评测流水线，亦支持 Eager 进程内同步模式便于单机演示；数据层采用 SQLAlchemy ORM，开发环境可用 SQLite，完整环境使用 PostgreSQL，并通过 Alembic 管理迁移。核心技术特点包括：评测领域状态机（创建、排队、运行、评判、完成、失败、取消、超时等合法迁移）、Agent 执行器与工具沙箱一体化执行轨迹（Trace）记录、规则评分与大模型评判混合策略、异步编排与结果聚合，以及安全中间件与租户过滤。上述系统架构设计、核心算法编码、领域模型定义及全流程测试验证，均由著作权人李凯昕独立完成，形成可运行、可观测、可评分的原创软件表达，而非对单一开源组件的简单堆叠调用。

应用价值

本软件将 Agent 质量评估从零散脚本与人工肉眼对比，升级为可配置、可审计、可回归的工程化工作台，显著降低企业引入或迭代 Agent 时的质检成本，提升问题定

位效率与评测结果可信度，适用于软件著作权登记材料展示、开源演示及企业内部 **Agent** 质量回归场景。

材料二：核心源代码（鉴别材料）

材料二：核心源代码（鉴别材料专用）

说明： 下列代码摘自本软件核心业务模块（用户鉴权、租户隔离、领域模型、规则指标、混合评分、工具沙箱、异步评测编排、任务 API），已去除第三方库源码、编译产物与敏感配置；密钥与地址统一使用占位符 `your_api_key_here`、`https://api.your-llm-endpoint.example/v1`。版权年份为 **2026**，与申请表开发完成日期（2026 年 7 月 14 日）所属年份一致。术语与说明书统一：评测任务（Task）、测试用例（TestSuite）、执行轨迹（Trace）、指标分（MetricScore）。

模块 A：API Key 鉴权（app/core/security.py）

```
# (c) 2026 AgentFlow-Eval / 李凯昕
"""API Key authentication helpers."""

from __future__ import annotations

import hmac
import secrets
from dataclasses import dataclass

from fastapi import Header, Request
from fastapi.security.utils import get_authorization_scheme_param

from app.config import settings
from app.utils.exceptions import AgentFlowError

class UnauthorizedError(AgentFlowError):
    def __init__(self, message: str = "Unauthorized") -> None:
        super().__init__(message=message, status_code=401, detail=None)

@dataclass(frozen=True)
class AuthIdentity:
    """Authenticated principal derived from API key."""
    key_id: str
    actor: str
    raw_key_prefix: str
```

```

def parse_api_keys(raw: str | list[str] | None) -> dict[str, str]:
    """Parse API keys into {secret: actor_name}."""
    if not raw:
        return {}
    if isinstance(raw, list):
        items = [str(x).strip() for x in raw if str(x).strip()]
    else:
        items = [p.strip() for p in str(raw).split(",") if p.strip()]
    mapping: dict[str, str] = {}
    for idx, item in enumerate(items, start=1):
        if ":" in item:
            secret, actor = item.split(":", 1)
            secret, actor = secret.strip(), actor.strip() or f"key_{idx}"
        else:
            secret, actor = item, f"key_{idx}"
        if secret:
            mapping[secret] = actor
    return mapping

def extract_api_key(
    request: Request,
    authorization: str | None = None,
    x_api_key: str | None = None,
) -> str | None:
    key = (x_api_key or request.headers.get("X-API-Key") or "").strip()
    if key:
        return key
    header = authorization or request.headers.get("Authorization")
    if header:
        scheme, param = get_authorization_scheme_param(header)
        if scheme.lower() == "bearer" and param:
            return param.strip()
        if scheme.lower() in {"apikey", "api-key"} and param:
            return param.strip()
    return None

def authenticate_api_key(api_key: str | None) -> AuthIdentity | None:
    if not api_key:
        return None
    keys = parse_api_keys(settings.API_KEYS)
    for secret, actor in keys.items():
        if hmac.compare_digest(api_key, secret):
            return AuthIdentity(
                key_id=secrets.token_hex(4),
                actor=actor,
                raw_key_prefix=api_key[:4] + "****",
            )
    return None

def require_auth_if_enabled(
    request: Request,
    authorization: str | None = Header(default=None),
    x_api_key: str | None = Header(default=None, alias="X-API-Key"),
) -> AuthIdentity | None:
    if not settings.AUTH_ENABLED:
        return AuthIdentity(key_id="dev", actor="anonymous", raw_key_prefix="dev")
    api_key = extract_api_key(request, authorization, x_api_key)
    identity = authenticate_api_key(api_key)
    if identity is None:
        raise UnauthorizedError("Invalid or missing API key")
    request.state.actor = identity.actor

```

```
request.state.auth = identity
return identity
```

模块 B：轻量多租户（app/core/tenancy.py）

```
# (c) 2026 AgentFlow-Eval / 李凯昕
"""Lightweight multi-tenant helpers keyed by API-key actor."""

from __future__ import annotations

from sqlalchemy import Select
from app.config import settings
from app.models.task import Task
from app.models.test_suite import TestSuite
from app.models.trace import Trace
from app.utils.exceptions import NotFoundError

def tenancy_enforced() -> bool:
    return bool(settings.AUTH_ENABLED or settings.TENANCY_ENABLED)

def admin_actors() -> set[str]:
    raw = settings.ADMIN_ACTORS or ""
    return {p.strip() for p in raw.split(",") if p.strip()}

def is_admin(actor: str | None) -> bool:
    return bool(actor) and actor in admin_actors()

def can_access_task(task: Task, actor: str | None) -> bool:
    """评测任务（Task）访问控制。"""
    if not tenancy_enforced():
        return True
    if is_admin(actor):
        return True
    owner = getattr(task, "created_by", None) or "anonymous"
    return owner == (actor or "anonymous")

def ensure_task_access(task: Task | None, actor: str | None, task_id: str = "") -> Task:
    if task is None or not can_access_task(task, actor):
        raise NotFoundError("Task", task_id or (task.id if task else ""))
    return task

def apply_owner_filter(query: Select, actor: str | None) -> Select:
    if not tenancy_enforced() or is_admin(actor):
        return query
    return query.where(Task.created_by == (actor or "anonymous"))

def apply_trace_owner_filter(query: Select, actor: str | None) -> Select:
    """执行轨迹（Trace）按所属评测任务（Task）所有者过滤。"""
    if not tenancy_enforced() or is_admin(actor):
```

```

        return query
    owner = actor or "anonymous"
    return (
        query.join(TestSuite, Trace.test_suite_id == TestSuite.id)
        .join(Task, TestSuite.task_id == Task.id)
        .where(Task.created_by == owner)
    )

```

模块 C：评测任务状态机与领域模型（app/models/task.py）

```

# (c) 2026 AgentFlow-Eval / 李凯昕
"""评测任务（Task）模型 — 一次评测的顶层聚合根。"""

import enum
from typing import TYPE_CHECKING
from sqlalchemy import Boolean, JSON, Enum, String, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship
from app.models.base import Base, PKMixin, TimestampMixin

if TYPE_CHECKING:
    from app.models.test_suite import TestSuite

class TaskStatus(str, enum.Enum):
    CREATED = "created"
    QUEUED = "queued"
    RUNNING = "running"
    WAITING_TOOL = "waiting_tool"
    JUDGING = "judging"
    COMPLETED = "completed"
    FAILED = "failed"
    CANCELLED = "cancelled"
    TIMEOUT = "timeout"

    @classmethod
    def allowed_transitions(cls) -> dict["TaskStatus", set["TaskStatus"]]:
        return {
            cls.CREATED: {cls.QUEUED, cls.CANCELLED},
            cls.QUEUED: {cls.RUNNING, cls.CANCELLED, cls.TIMEOUT},
            cls.RUNNING: {
                cls.WAITING_TOOL, cls.JUDGING, cls.FAILED,
                cls.CANCELLED, cls.TIMEOUT,
            },
            cls.WAITING_TOOL: {cls.RUNNING, cls.FAILED, cls.CANCELLED, cls.TIMEOUT},
            cls.JUDGING: {cls.COMPLETED, cls.FAILED, cls.CANCELLED},
            cls.COMPLETED: set(),
            cls.FAILED: set(),
            cls.CANCELLED: set(),
            cls.TIMEOUT: set(),
        }

    def can_transition_to(self, target: "TaskStatus") -> bool:
        return target in self.allowed_transitions().get(self, set())

class Task(PKMixin, TimestampMixin, Base):
    __tablename__ = "tasks"
    name: Mapped[str] = mapped_column(String(255), nullable=False)
    description: Mapped[str] = mapped_column(Text, nullable=False, default="")
    status: Mapped[TaskStatus] = mapped_column(

```



```

    Enum(
        TaskStatus, name="task_status", native_enum=False, length=20,
        values_callable=lambda e: [x.value for x in e],
    ),
    nullable=False, default=TaskStatus.CREATED,
)
agent_config: Mapped[dict] = mapped_column(JSON, nullable=False, default=dict)
celery_task_id: Mapped[str | None] = mapped_column(String(255), nullable=True)
is_archived: Mapped[bool] = mapped_column(Boolean, nullable=False, default=False)
created_by: Mapped[str] = mapped_column(String(100), nullable=False, default="anonymous")
test_suites: Mapped[list["TestSuite"]] = relationship(
    back_populates="task", cascade="all, delete-orphan",
)

```

模块 D：规则指标（app/core/judge_engine/metrics.py）

```

# (c) 2026 AgentFlow-Eval / 李凯昕
"""无 LLM 调用的轻量级辅助评分（支撑指标分 MetricScore 预打分）。"""

import re
from typing import Any

def calc_tool_accuracy(
    actual_tool_names: list[str],
    expected_tools: list[str],
) -> tuple[float, str]:
    actual_set = set(actual_tool_names)
    expected_set = set(expected_tools)
    if not expected_set:
        return 100.0, "无预期工具要求，默认满分。"
    missing = expected_set - actual_set
    extra = actual_set - expected_set
    penalty = 0.0
    reasons: list[str] = []
    if missing:
        penalty += len(missing) * 10.0
        reasons.append("缺少工具: " + ", ".join(sorted(missing)))
    if extra:
        penalty += len(extra) * 10.0
        reasons.append("多余工具: " + ", ".join(sorted(extra)))
    score = max(0.0, 100.0 - penalty)
    reason = "; ".join(reasons) if reasons else "全部预期工具均已调用，无多余调用。"
    return score, reason

def extract_answer_text(steps: list[dict[str, Any]]) -> str:
    """从执行轨迹（Trace）步骤数组中提取最终答案文本。"""
    for step in reversed(steps):
        if step.get("role") == "assistant":
            content = step.get("content", "")
            if content:
                return content
        if step.get("type") == "final_answer":
            return step.get("content", "")
    return ""

def _normalize(text: str) -> str:
    text = re.sub(r"\s+", " ", text).strip().lower()

```

```
return re.sub(r"^\w\u4e00-\u9fff]", "", text)
```

模块 E：混合评分引擎（app/core/judge_engine/llm_judge.py 核心）

```
# (c) 2026 AgentFlow-Eval / 李凯昕
"""Hybrid rule-based + LLM-as-Judge scoring -> 指标分 (MetricScore) 。"""

import hashlib
import json
import logging
import os
from openai import AsyncOpenAI
from app.core.judge_engine.base import BaseJudge
from app.core.judge_engine.metrics import calc_tool_accuracy, extract_answer_text

logger = logging.getLogger(__name__)
DIMENSION_WEIGHTS = {
    "tool_accuracy": 40.0,
    "answer_correctness": 40.0,
    "reasoning_coherence": 20.0,
}

class LLMJudge(BaseJudge):
    def __init__(self, api_key=None, base_url=None, model="gpt-4o-mini"):
        api_key = api_key or os.environ.get("OPENAI_API_KEY", "your_api_key_here")
        self.has_api_key = bool(api_key) and api_key != "your_api_key_here"
        if self.has_api_key:
            self.client = AsyncOpenAI(
                api_key=api_key,
                base_url=base_url or "https://api.your-llm-endpoint.example/v1",
            )
        else:
            self.client = None
        self.model = model
        self._cache: dict = {}

    async def evaluate(self, trace_steps, expected_output, expected_tools=None):
        expected_tools = expected_tools or []
        raw = json.dumps(
            {"steps": trace_steps, "output": expected_output, "tools": sorted(expected_tools)},
            sort_keys=True, ensure_ascii=False,
        )
        cache_key = hashlib.sha256(raw.encode("utf-8")).hexdigest()
        if cache_key in self._cache:
            return self._cache[cache_key]
        result = await self._do_evaluate(trace_steps, expected_output, expected_tools)
        if len(self._cache) > 128:
            self._cache.clear()
        self._cache[cache_key] = result
        return result

    def _pre_score(self, steps, expected_output, expected_tools):
        actual_tools = self._extract_tool_names(steps)
        tool_pct, tool_reason = calc_tool_accuracy(actual_tools, expected_tools)
        tool_score = round(tool_pct * 0.4, 1)
        extracted = self._extract_final_answer(steps)
        answer_score, answer_reason = self._lexical_answer_score(extracted, expected_output)
```

```

        coh_score, coh_reason = self._heuristic_coherence_score(steps)
    return {
        "scores": {
            "tool_accuracy": tool_score,
            "answer_correctness": answer_score,
            "reasoning_coherence": coh_score,
        },
        "total": tool_score + answer_score + coh_score,
        "reason": f"[Rule-based]
Tool:{tool_reason}|Answer:{answer_reason}|Coh:{coh_reason}",
        "token_cost": 0,
        "mode": "rule_only",
    }

    async def _do_evaluate(self, steps, expected_output, expected_tools):
        pre = self._pre_score(steps, expected_output, expected_tools)
        if self.has_api_key and self.client:
            try:
                return await self._llm_refine(steps, expected_output, expected_tools, pre)
            except Exception as exc:
                logger.warning("LLM refine failed: %s", exc)
                return pre
        return pre

    @staticmethod
    def _extract_tool_names(steps):
        names, seen = [], set()
        for s in steps:
            n = s.get("action") or s.get("tool_name") or ""
            if n and n not in seen and n != "final_answer":
                seen.add(n)
                names.append(n)
        return names

    @staticmethod
    def _extract_final_answer(steps):
        for s in reversed(steps):
            if s.get("action") == "final_answer":
                return s.get("action_input") or s.get("observation") or ""
            if s.get("type") == "final_answer":
                return s.get("content") or s.get("observation") or ""
        return extract_answer_text(steps)

```

模块 F：工具沙箱（app/core/agent_runner/tool_sandbox.py

摘要）

```

# (c) 2026 AgentFlow-Eval / 李凯昕
"""Safe built-in tool registry and sandboxed execution."""

from __future__ import annotations
import ast
import operator as op
from concurrent.futures import ThreadPoolExecutor, TimeoutError as FuturesTimeout
from typing import Any

_EXECUTOR = ThreadPoolExecutor(max_workers=4, thread_name_prefix="tool-sandbox")
DEFAULT_TOOL_TIMEOUT_SEC = 3.0
MAX_OUTPUT_CHARS = 4000

```

```

def tool_calculator(expression: str = "", **kwargs: Any) -> str:
    expr = (expression or kwargs.get("expr") or "").strip()
    if not expr or len(expr) > 200:
        return "Error: invalid expression"
    allowed_ops = {
        ast.Add: op.add, ast.Sub: op.sub, ast.Mult: op.mul,
        ast.Div: op.truediv, ast.Pow: op.pow, ast.USub: op.neg,
        ast.UAdd: op.pos, ast.Mod: op.mod,
    }

    def _eval(node: ast.AST) -> float:
        if isinstance(node, ast.Expression):
            return _eval(node.body)
        if isinstance(node, ast.Constant) and isinstance(node.value, (int, float)):
            return float(node.value)
        if isinstance(node, ast.BinOp):
            fn = allowed_ops[type(node.op)]
            return fn(_eval(node.left), _eval(node.right))
        if isinstance(node, ast.UnaryOp):
            return allowed_ops[type(node.op)](_eval(node.operand))
        raise ValueError("unsupported")

    tree = ast.parse(expr, mode="eval")
    for n in ast.walk(tree):
        if isinstance(n, (ast.Name, ast.Call, ast.Attribute, ast.Subscript)):
            raise ValueError("Only numeric expressions are allowed")
    return f"Result: {_eval(tree)}"

def run_tool_sandboxed(name: str, fn, kwargs: dict, timeout: float = DEFAULT_TOOL_TIMEOUT_SEC)
-> str:
    future = _EXECUTOR.submit(fn, **(kwargs or {}))
    try:
        out = str(future.result(timeout=timeout))
    except FuturesTimeout:
        return f"Error: tool {name} timed out after {timeout}s"
    except Exception as exc:
        return f"Error: tool {name} failed: {exc}"
    if len(out) > MAX_OUTPUT_CHARS:
        out = out[:MAX_OUTPUT_CHARS] + "...[truncated]"
    return out

```

模块 G：异步评测编排 (app/core/celery_app/tasks.py 摘要)

```

# (c) 2026 AgentFlow-Eval / 李凯昕
"""Celery async evaluation pipeline: Task -> TestSuite -> Trace -> MetricScore."""

from __future__ import annotations
import logging
from typing import Any
from sqlalchemy import select
from app.core.celery_app.celery import celery_app
from app.core.dependencies import async_session_factory
from app.models.task import Task, TaskStatus
from app.models.test_suite import TestSuite
from app.models.trace import Trace, TraceStatus

logger = logging.getLogger(__name__)

```

```

def build_agent_runner(agent_config: dict[str, Any] | None = None):
    from app.config import settings as app_settings
    from app.core.agent_runner.openai_runner import OpenAIREActRunner
    cfg = agent_config or {}
    return OpenAIREActRunner(
        api_key=app_settings.OPENAI_API_KEY or "your_api_key_here",
        base_url=app_settings.OPENAI_BASE_URL or "https://api.your-llm-endpoint.example/v1",
        model=cfg.get("model", "gpt-4o-mini"),
        max_iterations=cfg.get("max_iterations", 5),
    )

@celery_app.task(bind=True, max_retries=3, soft_time_limit=300, time_limit=320)
def run_single_test_suite(self, test_suite_id: str, agent_config: dict) -> dict:
    """执行单条测试用例（TestSuite），落库执行轨迹（Trace）。"""

    async def _execute():
        async with async_session_factory() as session:
            suite = (
                await session.execute(select(TestSuite).where(TestSuite.id == test_suite_id))
            ).scalar_one_or_none()
            if not suite:
                raise ValueError(f"TestSuite not found: {test_suite_id}")
            from app.core.agent_runner.tool_sandbox import resolve_tools_for_suite
            runner = build_agent_runner(agent_config or {})
            tools = resolve_tools_for_suite(suite.expected_tools)
            agent_result = await runner.run(query=suite.user_query, tools=tools)
            status_map = {
                "success": TraceStatus.SUCCESS,
                "max_iterations_reached": TraceStatus.FAILED,
                "failed": TraceStatus.FAILED,
            }
            trace = Trace(
                test_suite_id=test_suite_id,
                user_query=suite.user_query,
                steps=agent_result.get("steps", []),
                total_tokens=agent_result.get("total_tokens", 0),
                response_time_ms=agent_result.get("response_time_ms", 0),
                status=status_map.get(agent_result.get("status", ""), TraceStatus.FAILED),
            )
            session.add(trace)
            await session.commit()
            await session.refresh(trace)
            return {"trace_id": trace.id, "status": trace.status.value}

    return _run_async(_execute)

```

模块 H：评测任务列表 API（app/api/v1/endpoints/tasks.py 摘要）

```

# (c) 2026 AgentFlow-Eval / 李凯昕
"""评测任务（Task）列表 API，含 Actor 租户过滤。"""

from fastapi import APIRouter, Depends, Query, Request
from sqlalchemy import func, select
from sqlalchemy.ext.asyncio import AsyncSession
from app.core.dependencies import get_db
from app.core.tenancy import apply_owner_filter

```

```

from app.models.task import Task
from app.models.test_suite import TestSuite
from app.schemas.task import TaskListResponse, TaskResponse

router = APIRouter()

def _actor(request: Request) -> str:
    return getattr(request.state, "actor", None) or "anonymous"

@router.get("", response_model=TaskListResponse)
async def list_tasks(
    request: Request,
    page: int = Query(1, ge=1),
    page_size: int = Query(20, ge=1, le=100),
    status: str | None = Query(None),
    include_archived: bool = Query(False),
    session: AsyncSession = Depends(get_db),
):
    actor = _actor(request)
    query = select(Task)
    count_query = select(func.count(Task.id))
    if not include_archived:
        query = query.where(Task.is_archived.is_(False))
        count_query = count_query.where(Task.is_archived.is_(False))
    if status:
        query = query.where(Task.status == status)
        count_query = count_query.where(Task.status == status)
    query = apply_owner_filter(query, actor)
    count_query = apply_owner_filter(count_query, actor)
    total = (await session.execute(count_query)).scalar() or 0
    rows = (
        await session.execute(
            query.order_by(Task.created_at.desc())
                .offset((page - 1) * page_size)
                .limit(page_size)
        )
    ).scalars().all()
    items = []
    for task in rows:
        suite_count = (
            await session.execute(
                select(func.count(TestSuite.id)).where(TestSuite.task_id == task.id)
            )
        ).scalar() or 0
        items.append(
            TaskResponse(
                id=task.id,
                name=task.name,
                description=task.description,
                status=task.status.value,
                agent_config=task.agent_config or {},
                test_suite_count=suite_count,
                created_by=getattr(task, "created_by", None) or "anonymous",
            )
        )
    return TaskListResponse(items=items, total=total, page=page, page_size=page_size)

```

(连续完整源程序请以 `scripts/export-soft-copyright.ps1` 导出的前 30 页与后 30 页为准；本材料侧重核心业务逻辑鉴别。)

材料三：用户使用手册（操作说明书）

材料三：用户使用手册（操作说明书）

软件名称： AgentFlow-Eval Agent 自动化评测工作台（界面名称： AgentFlow Intelligence/驾驶舱）

版本号： V1.0

著作权人： 李凯昕

开发完成日期： 2026 年 7 月 14 日

文档类型： 用户操作手册

软件著作权登记名称为「AgentFlow-Eval Agent 自动化评测工作台」，前端工作台显示名称为「AgentFlow Intelligence / 驾驶舱」，二者指向同一软件产品。

1. 软件概述

AgentFlow Intelligence（驾驶舱）是一款面向企业 AI 场景的 Agent 自动化评测 Web 工作台。用户可创建评测任务（Task）、导入业务测试用例（TestSuite），系统自动调度 Agent 执行，记录完整执行轨迹（Trace），并结合规则指标与大模型评审（LLM-as-Judge）生成多维指标分（MetricScore）与结构化报告。

主要使用角色： AI 应用开发者、算法评测人员、质检人员。

访问方式： 浏览器访问前端工作台（推荐 Chrome / Edge 近两个大版本）。

核心业务闭环：

1. 创建评测任务（Task）并配置 Agent 参数

- 2. 导入或维护测试用例（TestSuite）
 - 3. 执行评测（异步或本地 Eager）
 - 4. 查看执行轨迹（Trace）的 DAG、步骤日志与指标分（MetricScore）
 - 5. 查看聚合报告，必要时人工复核改分
-

2. 运行环境

2.1 硬件环境（建议）

项目	最低建议
CPU	2 核
内存	4 GB
磁盘	10 GB 可用空间
显示	1280×720 及以上

2.2 软件环境

组件	版本要求
操作系统	Windows 10/11、Linux 或 macOS
Python	3.11 及以上
Node.js	18 及以上（前端开发模式）
浏览器	Chrome / Edge 近两个大版本
Docker	可选，用于完整栈（PostgreSQL + Redis + Celery）

2.3 外部依赖说明

- 大模型服务：需配置兼容 OpenAI 协议的 API 地址与密钥（占位符：`your_api_key_here`、`https://api.your-llm-endpoint.example/v1`）。
 - 完整异步模式需要 Redis 与 Celery Worker；本地演示可将 `CELERY_TASK_ALWAYS_EAGER=true` 设为进程内同步执行。
-

3. 安装与启动

3.1 方式 A：本地 Eager（最小依赖）

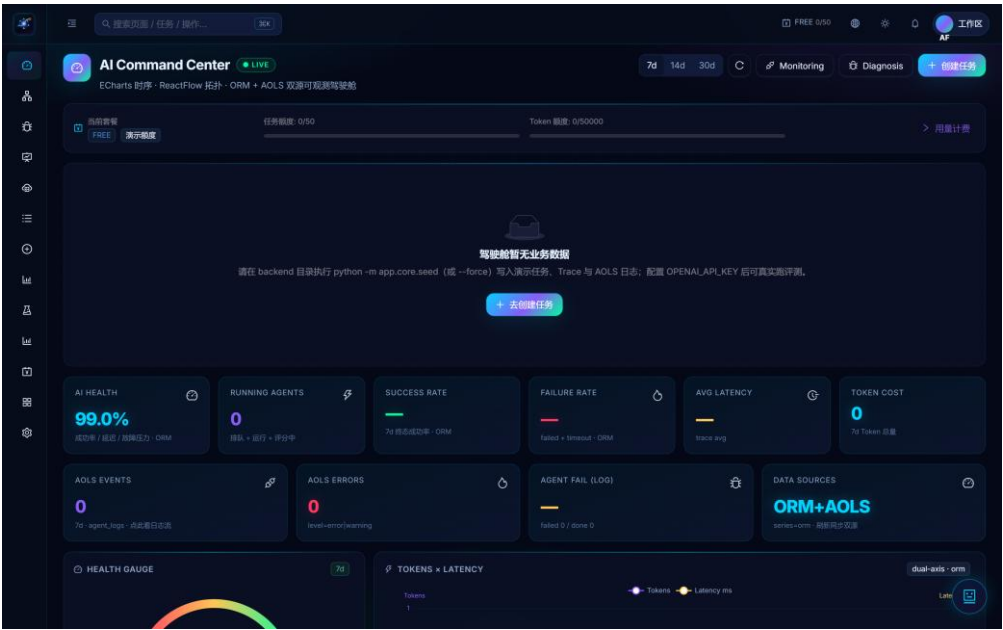
步骤 1：后端

1. 进入 `backend` 目录，创建并激活 Python 虚拟环境。
2. 执行 `pip install -r requirements.txt`。
3. 复制环境模板为 `.env`，设置：
 - `OPENAI_API_KEY=your_api_key_here`
 - `OPENAI_BASE_URL=https://api.your-llm-endpoint.example/v1`（如需）
 - `CELERY_TASK_ALWAYS_EAGER=true`
4. 启动： `uvicorn app.main:app --reload --port 8000`。
5. 浏览器访问 `http://127.0.0.1:8000/health` 确认服务健康。

步骤 2：前端

1. 进入 `frontend` 目录，执行 `npm install`。
2. 配置 `VITE_API_BASE_URL=http://127.0.0.1:8000/api/v1`。

- 3. 执行 `npm run dev`。
- 4. 浏览器访问 `http://localhost:5173`。



【截图：驾驶舱看板图】

图 1：驾驶舱看板图（command_center.png）

3.2 方式 B：Docker 全栈

- 1. 生成 Docker 环境文件（参考 `scripts/generate-deploy-env.ps1`）。
 - 2. 在 `backend/.env.docker` 填入 `OPENAI_API_KEY=your_api_key_here`。
 - 3. 执行 Docker Compose 启动全栈。
 - 4. 访问前端入口与 API 文档（如 `http://localhost/`、`http://localhost:8000/docs`）。
- API 文档（Swagger）界面可通过后端 `/docs` 路径访问；前端工作台访问根路径自动跳转至驾驶舱。

3.3 写入演示数据（可选）

在后端环境中执行：`python -m app.core.seed`，可生成示例评测任务（Task）与测试用例（TestSuite），便于熟悉界面。。

4. 系统导航结构

系统采用左-右布局，左侧导航栏按四大功能分组组织：

COMMAND（命令与控制）

- 驾驶舱（AI Command Center） — /dashboard
- Trace（轨迹回溯） — /traces
- 故障诊断 — /diagnosis
- 分析（分析中心） — /analytics
- 监控（运行监控） — /monitoring

EVALUATE（评测）

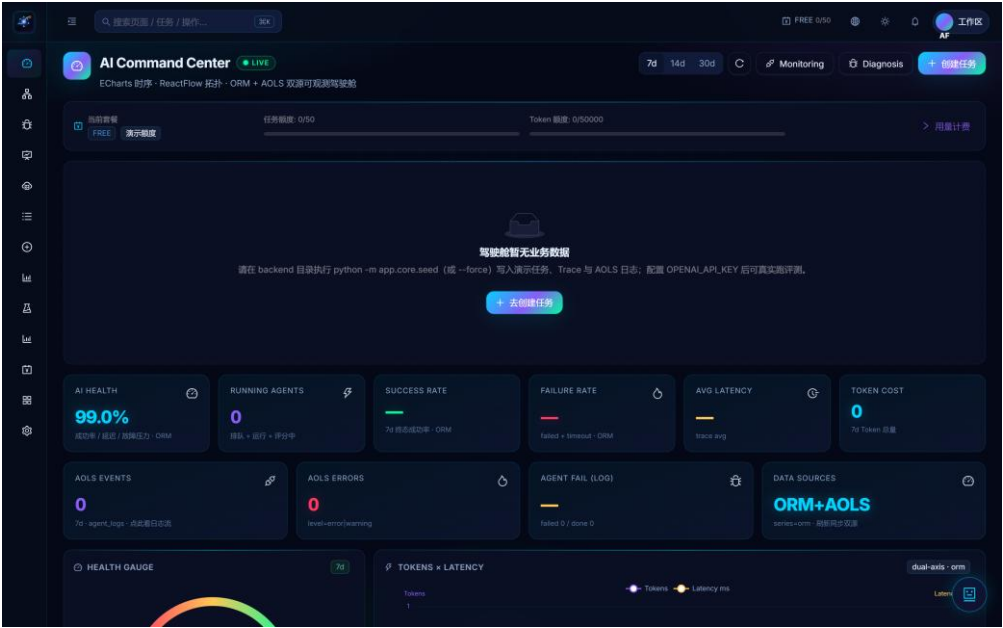
- 任务（评测任务列表） — /tasks
- 创建任务 — /tasks/create
- 报告 — /reports

OPERATE（运营）

- 用量计费 — /billing
- 插件市场 — /plugins

SYSTEM（系统）

- 设置中心 — /settings



【截图：导航结构图】

图 2：系统导航结构图（nav_overview.png）

5. 功能模块详解

5.1 驾驶舱（AI Command Center）

入口： 左侧导航 COMMAND 分组「驾驶舱」或路由 /dashboard。

驾驶舱为系统的核心监控界面，提供以下可视化组件：

六项核心指标卡（指标面板）

- **AI HEALTH** — 系统整体健康度评分
- **RUNNING AGENTS** — 当前正在运行的 Agent 数量
- **SUCCESS RATE** — 整体任务成功率

- **FAILURE RATE** — 整体任务失败率

- **AVG LATENCY** — 平均响应延迟

- **TOKEN COST** — 累计 Token 消耗

四项状态卡片区

- **AOLS EVENTS** — 事件聚合计数

- **AOLS ERRORS** — 错误聚合计数

- **AGENT FAIL** — Agent 失败统计

- **DATA SOURCES** — 数据源连接状态（ORM + AOLS）

HEALTH GAUGE 仪表盘 — 实时显示系统健康度指针。

TOKENS × LATENCY 双轴图 — 左侧纵轴为 Token 消耗量，右侧纵轴为延迟时间，横轴为时间线。

AGENT PIPELINE TOPOLOGY 五节点拓扑图

- 节点链：**User Request → Planner Agent → Tool Calling → Observation → LLM Judge**

- 各节点状态色标：

- 绿（**HEALTHY**） — 正常

- 黄（**DEGRADED**） — 降级

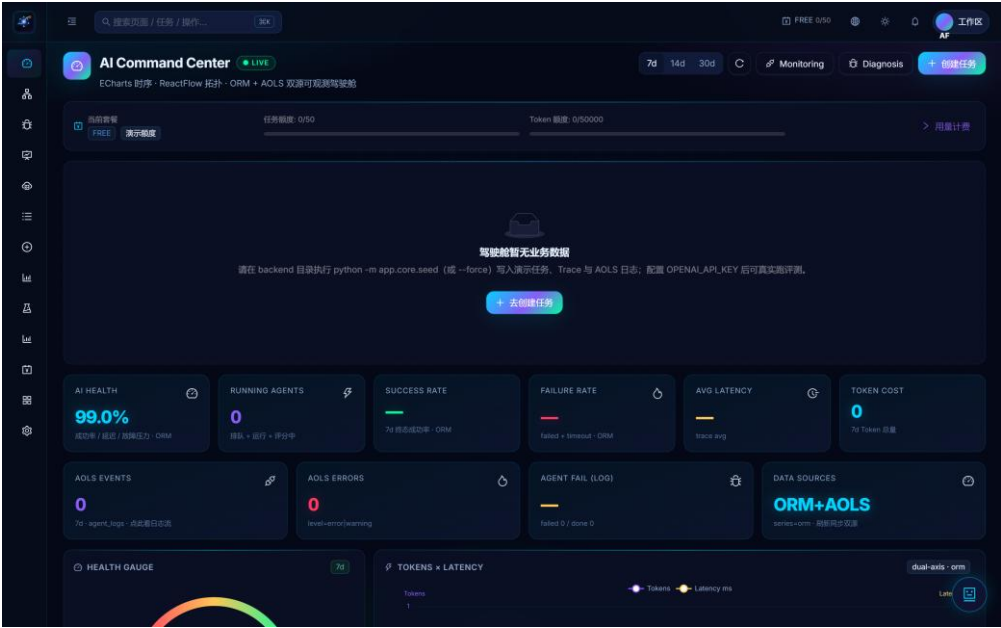
- 红（**FAILURE**） — 失败

- 灰（**IDLE**） — 空闲

右上角操作区

- 时间范围选择：**7d / 14d / 30d** 切换按钮

- 快捷入口：**Monitoring、Diagnosis、创建任务** 按钮



【截图：驾驶舱看板图】

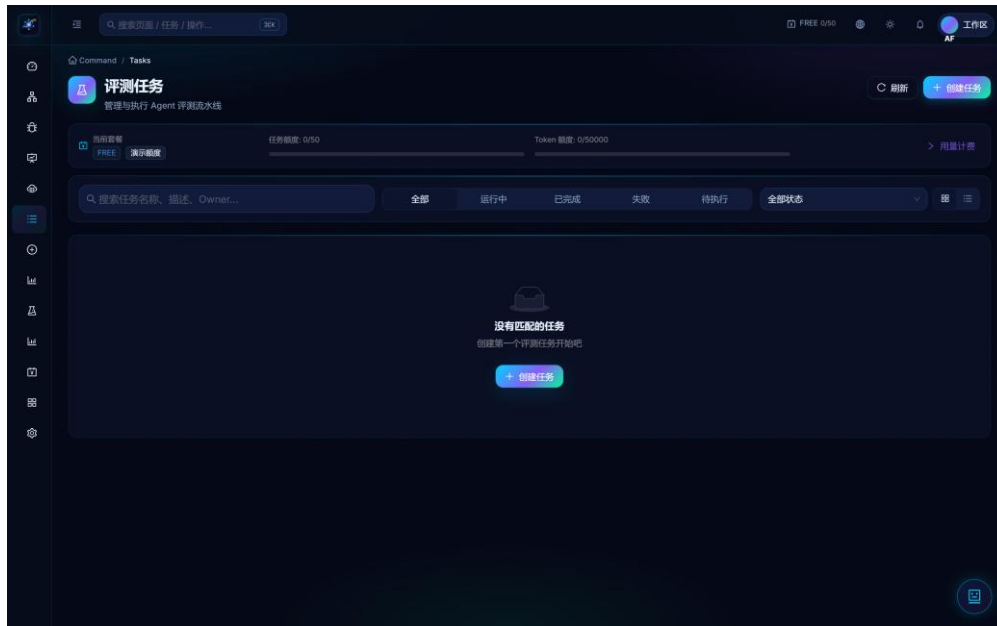
图 3：驾驶舱（AI Command Center）看板图（command_center.png）

5.2 评测任务列表

入口： 左侧导航 EVALUATE 分组「任务」或路由 /tasks。

交互闭环：

- 1. 点击「任务」→ 加载评测任务（Task）分页列表。
- 2. 可按状态筛选，切换是否包含已归档任务。
- 3. 点击某一任务名称 → 进入任务详情页。
- 4. 可对任务执行归档或删除（以界面按钮为准）。



【截图：任务列表】

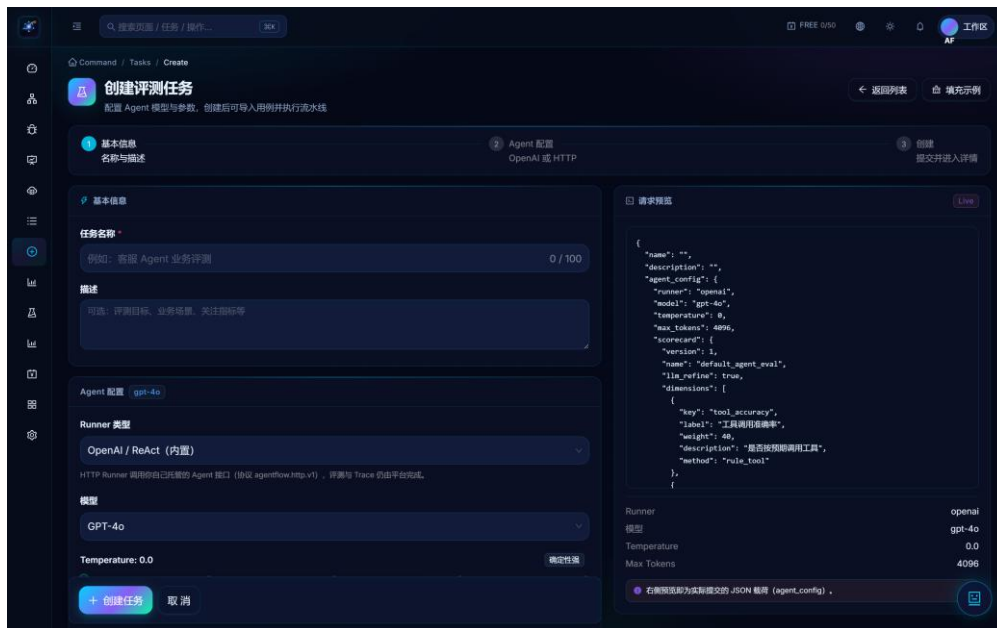
图 4：评测任务列表界面（task_list.png）

5.3 创建评测任务

入口：EVALUATE 分组「创建任务」或路由 /tasks/create。

交互闭环：

1. 点击「创建任务」→ 加载创建表单。
2. 输入任务名称、描述。
3. 配置 Agent 参数（如模型、最大迭代轮次等）。
4. 点击「保存/创建」→ 后端写入评测任务（Task）→ 跳转详情或列表并展示新任务。



【截图：创建任务】

图 5：创建评测任务表单界面（task_create.png）

5.4 测试用例维护与导入

入口：任务详情页用例区域。

交互闭环：

1. 打开任务详情 → 加载测试用例（TestSuite）列表。
2. 选择上传 CSV 或 JSON 文件 → 点击上传按钮。
3. 后端解析 user_query、expected_output、expected_tools 等字段并批量写入。
4. 界面刷新展示新增用例条数。

图 6：测试用例上传界面 — 请参考任务详情页的用例管理区域截图（文件：testcase_upload.png，如未单独截图则可参照 task_create 页面中的用例上传组件）。

5.5 执行评测

入口：任务详情页「执行」按钮。

交互闭环：

1. 确认测试用例（TestSuite）已就绪后点击「执行评测」。
2. 系统将评测任务（Task）状态置为排队/运行，并投递异步任务（或 Eager 同步执行）。
3. 前端通过刷新或 WebSocket 活动推送展示进度。
4. 执行完成后状态变为完成或失败，可进入执行轨迹（Trace）与报告查看。

5.6 运行监控

入口：左侧导航 COMMAND 分组「监控」或路由 /monitoring。

运行监控页面提供系统运行时的完整可观测性视图：

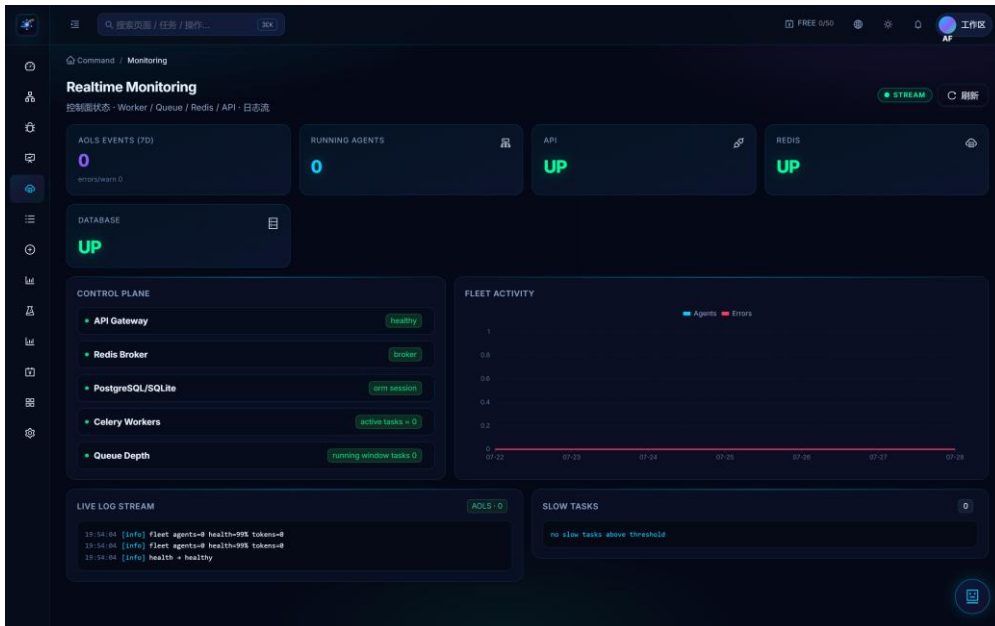
基础设施状态面板

- **API Gateway** — API 网关健康状态
- **Redis Broker** — Redis 消息代理连接状态
- **PostgreSQL** — 数据库连接状态
- **Celery Workers** — 异步任务 Worker 在线数与忙闲状态
- **Queue Depth** — 消息队列深度实时值

Fleet Activity（集群活动） — 各 Worker 节点当前处理的任务状态列表。

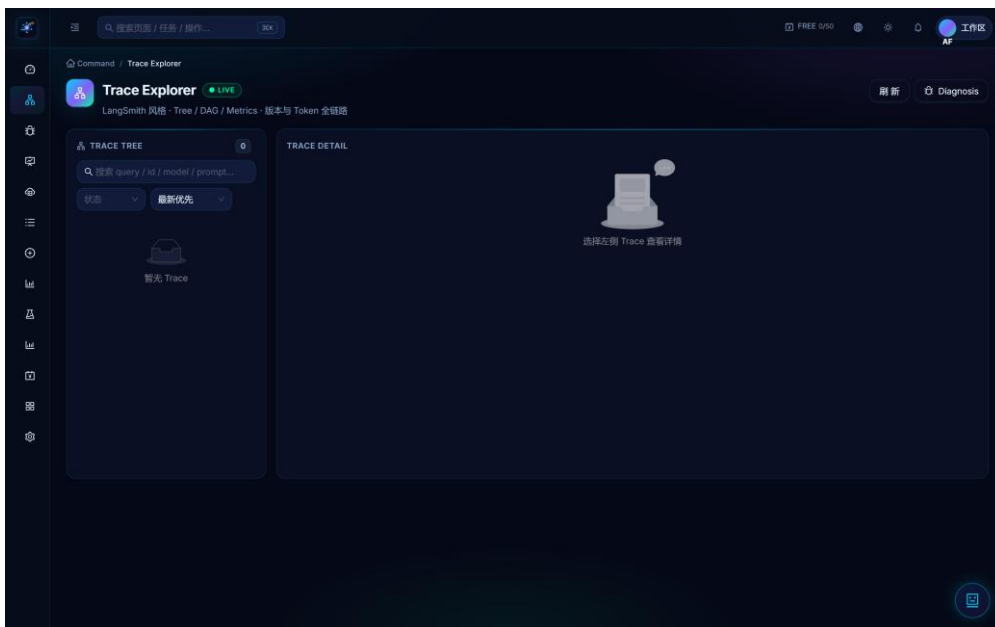
LIVE LOG STREAM（实时日志流） — 滚动输出的 Agent 运行日志流，支持按级别（INFO/WARN/ERROR）过滤。

SLOW TASKS（慢任务） — 超过延迟阈值的任务列表，展示任务名称、耗时与所属 Worker。



【截图：运行监控】

图7：运行监控界面（monitoring.png）



【截图：轨迹回溯】

图8：轨迹回溯—Trace Tree 与Trace Detail 界面（trace.png）

图9：轨迹回溯—Steps/DAG/Metrics 视图与步骤展开详情（同一截图已涵盖，trace.png）

5.7 轨迹回溯 (Trace)

入口：左侧导航 COMMAND 分组「Trace」或路由 /traces。

轨迹回溯页面提供执行过程的完整可视化追溯：

视图结构

- **TRACE TREE** — 左侧执行轨迹树，按时间线展示步骤层级
- **TRACE DETAIL** — 右侧详情面板，展示当前选中步骤的：
 - **Latency** — 步骤耗时
 - **Tokens** — Token 消耗量
 - **Steps** — 步骤数
 - **AVG SCORE** — 平均评分

视图切换标签

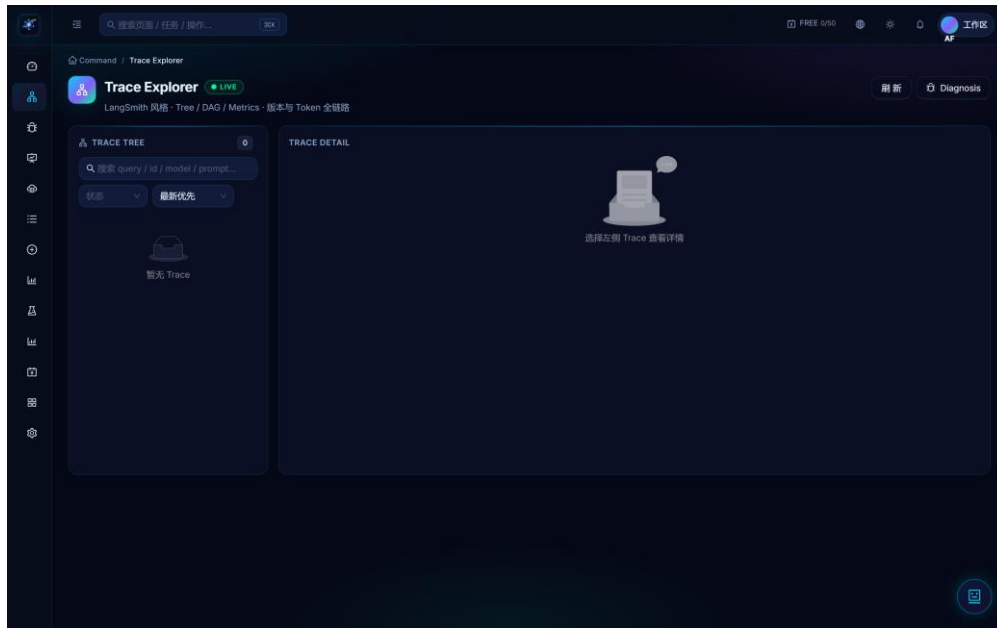
- **Steps** — 步骤列表视图
- **DAG** — ReactFlow 有向无环图视图
- **Metrics** — 指标分视图

Steps 展开详情 — 点击某一步骤可展开查看：

- **Prompt** — 发送给 LLM 的完整提示词
- **Input** — 步骤输入数据

操作按钮

- **诊断 (Diagnosis)** — 转入故障诊断分析
- **Judge (评判)** — 触发 LLM 重新评判



【截图：轨迹回溯】

图 8：轨迹回溯 — Trace Tree 与 Trace Detail 界面 (trace.png)

图 9：轨迹回溯 — Steps/DAG/Metrics 视图与步骤展开详情 (同一截图已涵盖, trace.png)

5.8 人工复核与重新评判

入口：执行轨迹（Trace）详情评分区域。

交互闭环：

1. 查看机评各维度指标分（MetricScore）与扣分说明。
2. 如需调整，输入人工分数与审核人标识 → 提交复核。
3. 系统以人工分覆盖有效分并记录审核标记。
4. 亦可触发重新 LLM 评判（在配置密钥可用时）。

5.9 分析中心

入口：左侧导航 COMMAND 分组「分析」或路由 /analytics。

分析中心提供评测数据的多维统计分析视图：

AGENT CAPABILITY RADAR（能力雷达图） — 六维能力评价：

- **Reasoning** — 推理能力
- **Accuracy** — 准确率
- **Safety** — 安全性
- **Cost** — 成本效率
- **Speed** — 响应速度
- **Tool Usage** — 工具调用能力

MODEL COMPARE（模型对比） — 不同模型在同一评测集上的得分对比柱状图。

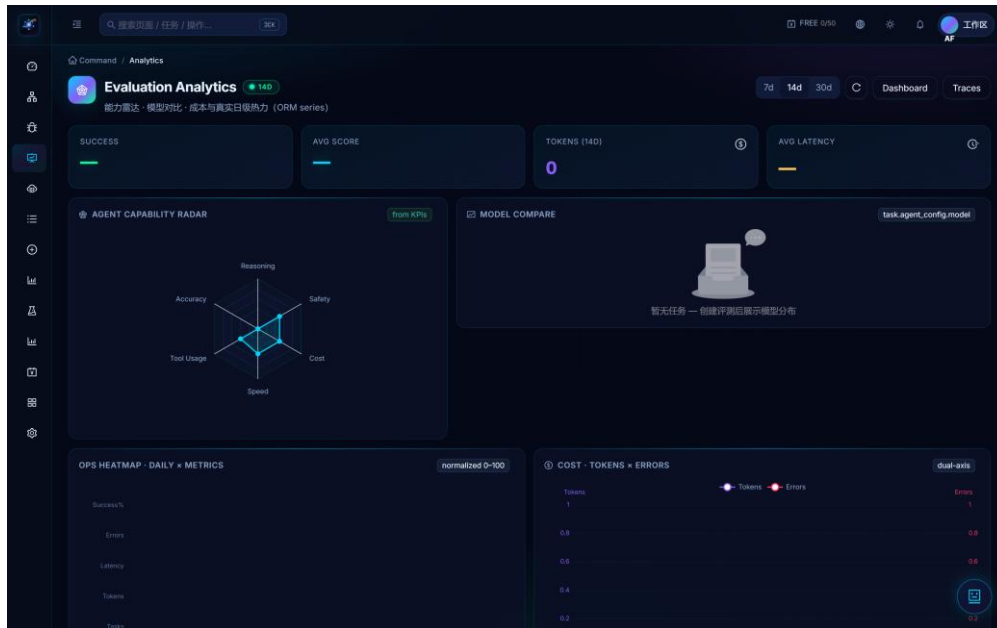
OPS HEATMAP（运维热力图） — 按时间与状态维度的错误/失败分布热力图。

COST 双轴图 — Token 消耗与延迟的双轴趋势图。

SCORE DISTRIBUTION（分数分布） — 各维度得分分布直方图。

四张核心指标卡

- **SUCCESS** — 总体成功率
- **AVG SCORE** — 平均综合得分
- **TOKENS** — 累计 Token 数
- **AVG LATENCY** — 平均延迟



【截图：分析中心】

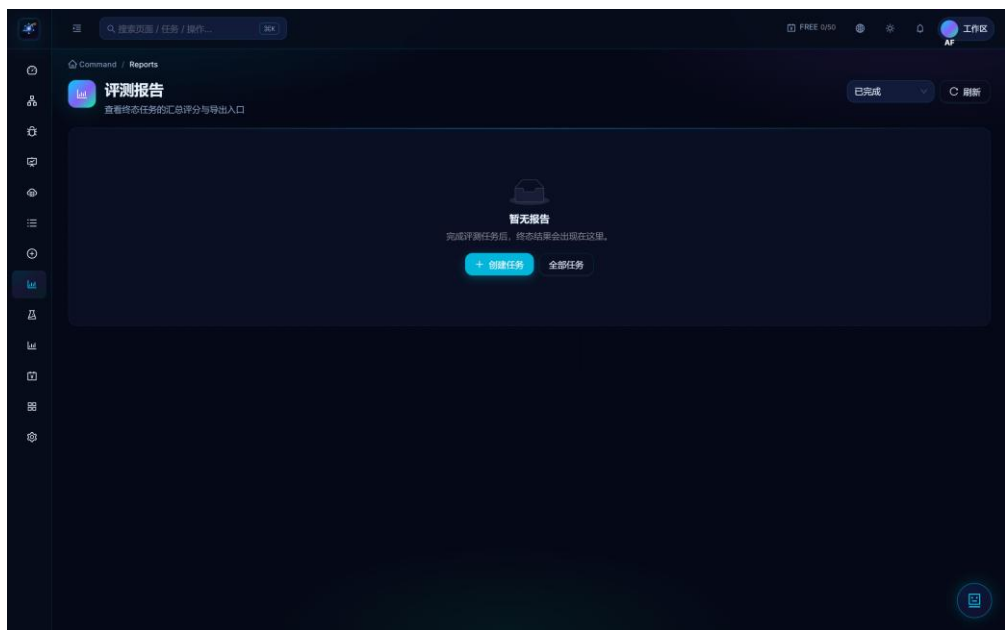
图 10：分析中心界面（analytics.png）

5.10 评测报告

入口： 左侧导航 EVALUATE 分组「报告」或路由 /reports，以及任务侧报告入口。

交互闭环：

1. 点击报告列表中的报告项 → 加载聚合结果。
2. 查看各测试用例（TestSuite）/ 维度指标分（MetricScore）分布与说明。
3. 可结合任务详情回溯具体执行轨迹（Trace）。



【截图：评测报告】

图 11：评测报告详情界面 (reports.png)

5.11 用量计费

入口：左侧导航 OPERATE 分组「用量计费」或路由 /billing。

用量计费页面展示 API Token 消耗统计与费用概览，包含：

- 当前周期 Token 使用量与预估费用
- 按模型分组的消耗明细表
- 历史周期费用趋势图

图 12：用量计费界面 (billing.png，如未单独截图可参照 analytics 页面的 Cost 双轴图)

5.12 插件市场

入口：左侧导航 OPERATE 分组「插件市场」或路由 /plugins。

插件市场页面展示可安装的第三方工具插件列表，支持在线安装/卸载与版本管理。
管理。

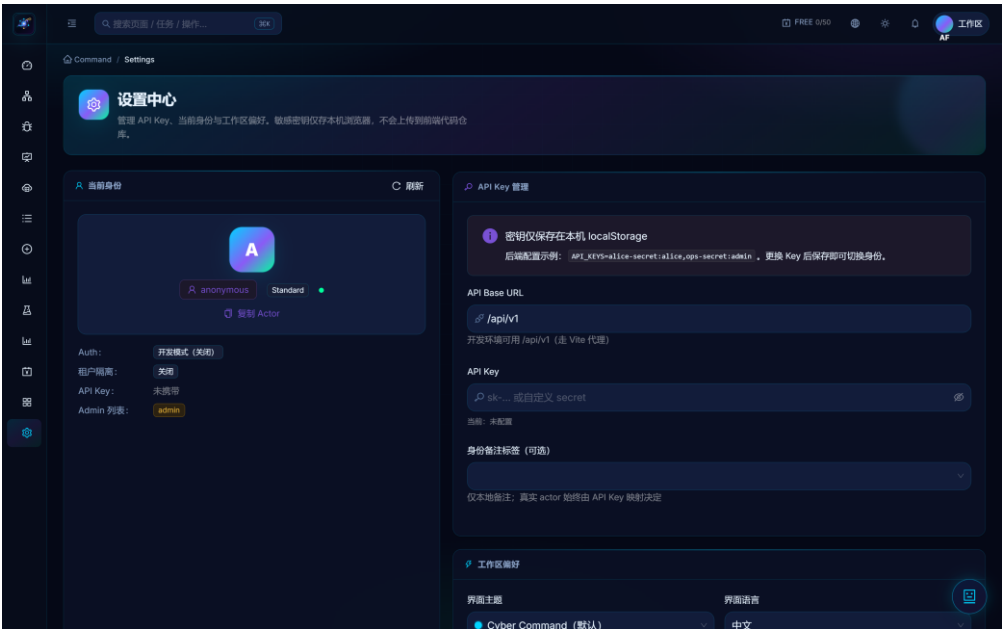
5.13 设置中心

入口： 左侧导航 SYSTEM 分组「设置中心」或路由 /settings。

设置中心提供以下配置管理功能：

交互闭环：

- 1. 打开设置页 → 加载主题、工作区与鉴权相关配置项。
- 2. 切换主题或保存偏好 → 界面即时生效。
- 3. 查看工具沙箱列表或探测结果（如界面提供）。
- 4. API 密钥管理与外部服务集成配置。



【截图：设置中心】

图 13：设置中心界面 (settings.png)

6. 典型操作流程（端到端）

- 1. 启动前后端服务并完成健康检查。

- 2. （可选）执行种子脚本生成示例数据（正式截图须替换为真实业务样式）。
- 3. 创建评测任务（Task）并配置 Agent。
- 4. 上传 CSV/JSON 测试用例（TestSuite）。
- 5. 点击执行评测，等待状态完成。
- 6. 在任务详情查看执行轨迹（Trace）的 DAG、步骤与指标分（MetricScore）。
- 7. 打开报告页查看聚合结果；必要时人工复核。
- 8. 对历史任务进行归档或删除，保持列表整洁。

7. 常见问题

问题	可能原因	处理建议
前端无法请求 API	地址或跨域配置错误	检查 `VITE_API_BASE_URL` 是否含 `/api/v1`
健康检查 redis 为 unavailable	未启动 Redis	本地 Eager 可忽略；完整模式请启动 Redis
执行后无执行轨迹（Trace）	密钥无效或测试用例（TestSuite）为空	检查 `OPENAI_API_KEY` 与用例列表
401 未授权	已开启鉴权但未传 Key	请求头增加 `X-API-Key: your_api_key_here`
执行卡住	异步模式未起 Worker	启动 Celery Worker，或设 Eager 为 true
评分仅为规则分	未配置有效 LLM Key	属正常降级；配置密钥后可 hybrid 精修

8. 版本信息

项目	内容
软件名称	AgentFlow-Eval Agent 自动化评测工作台
软件版本	V1.0
文档版本	V1.0（终稿修订版）
著作权人	李凯昕
开发完成日期	2026 年 7 月 14 日
主要技术	Python / FastAPI / Celery / React / TypeScript / Ant Design / ReactFlow
运行形态	B/S 前后端分离

9. 注意事项

1. 生产环境请关闭不必要的公开文档路径，并启用鉴权与租户隔离。
2. 密钥、数据库口令不得写入截图与提交材料明文，统一使用占位符。
3. 截图应体现真实业务数据样式，避免测试水印与 Demo 字样；文件名格式为「图 X-功能名称.png」。

材料四：软件设计说明书（技术文档）

材料四：软件设计说明书（技术文档）

软件名称： AgentFlow-Eval Agent 自动化评测工作台

版本号： V1.0

著作权人 / 开发人： 李凯昕

开发方式： 独立开发（独自开发）

开发完成日期： 2026 年 7 月 14 日

文档性质： 软件设计说明书（纯文字技术说明，不含大段代码）

第一章 引言

1.1 编写目的

本说明书用于完整描述 AgentFlow-Eval Agent 自动化评测工作台的总体设计、数据设计、核心算法、模块协作、接口规范与安全机制，作为中国版权保护中心软件著作权登记的文档鉴别材料之一，并作为后续维护与二次扩展的技术依据。文档重点阐明：系统架构设计、核心算法编码、领域模型定义及全流程测试验证均由著作权人李凯昕独立完成，充分支撑软件原创性主张。

1.2 项目背景

近年来，大语言模型在企业客服、办公自动化、知识检索与流程编排等场景快速落地。与单轮问答不同，Agent 需要在多轮交互中进行目标分解、工具选择、结果观察与再规划。业务侧真正关心的不仅是最终自然语言答案，更包括是否调用了

正确工具、是否出现多余调用、中间推理是否偏离业务约束。若仅依赖人工抽检或对最终文本做粗糙比对，将导致回归成本高、缺陷定位慢、结果不可追溯。

针对上述痛点，本软件将评测过程产品化为可运行的工作台：用户以测试用例（TestSuite）固化业务预期，系统自动调度 Agent 执行，完整记录 ReAct 风格的思考—行动—观察执行轨迹（Trace），再通过规则指标与大模型评判相结合的方式生成多维指标分（MetricScore）与报告，并提供前端可视化与审计能力。软件形态为 B/S 前后端分离应用，可在本地或容器环境完整运行。

1.3 设计目标

第一，工程化：以评测任务（Task）、测试用例（TestSuite）、执行轨迹（Trace）、指标分（MetricScore）为核心对象，建立清晰状态机与持久化模型。第二，可观测：结构化存储每一步推理与工具调用，并以链路图呈现。第三，可评分：规则基线分加可选 LLM 精修，并允许人工复核。第四，可扩展：执行器与评判器可替换，工具可注册。第五，可落地：鉴权、租户隔离、限流、审计与 WebSocket 活动推送。

1.4 适用范围

适用于软件著作权申请材料编制、系统验收说明、二次开发人员阅读以及运维部署参考。

1.5 术语与约定

本说明书及配套材料统一采用下列术语（代码标识附于括号内）：评测任务（Task）；测试用例（TestSuite）；执行轨迹（Trace）；指标分（MetricScore）；审计日志（AuditLog）；Agent 指多步推理并调用工具的智能体；ReAct 指推理—行动—观察循环；LLM-as-Judge 指利用大模型对轨迹语义评分；Eager 指进程内同步执行；Actor 指 API Key 映射的逻辑主体。接口路径相对于 /api/v1。

1.6 参考资料与项目边界

设计以仓库 `backend/app` 与 `frontend/src` 真实实现为准。开源基础组件按其许可证使用；登记对象为李凯昕独创的业务程序表达、领域模型、流程编排与界面交互设计，而非第三方库源码或大模型权重。

第二章 总体架构设计

2.1 架构风格与部署形态

系统采用前后端分离三层架构：表现层负责路由、状态、可视化；业务服务层负责鉴权、编排、执行、评分与审计；数据层负责关系库与队列。部署支持本地 SQLite+Eager 最小形态，以及 PostgreSQL+Redis+Celery 的完整形态。著作权人李凯昕特别强调同一业务代码路径兼容两种运行模式，避免演示与生产分叉。

2.2 逻辑子系统划分

接入子系统、评测业务子系统、Agent 运行子系统、评判子系统、运维支撑子系统、前端工作台子系统。六者通过清晰契约协作，构成端到端闭环。

2.3 分层职责

表现层不直连数据库；路由层保持薄封装；核心算法集中在 `agent_runner` 与 `judge_engine`；异步编排集中在 `celery_app`；数据层以 SQLAlchemy 表达 Task—TestSuite—Trace—MetricScore 聚合。

2.4 关键技术选型说明

FastAPI 提供异步与 OpenAPI；Celery/Redis 剥离长耗时调用；SQLAlchemy 2.0 异步会话与 FastAPI 一致；React/TypeScript 支撑复杂交互；Ant Design 构建企业工作台；ReactFlow 表达执行步骤有向图。选型服务于可运行、可观测、可评分目标。

2.5 主数据路径

用户创建评测任务（Task）并导入测试用例（TestSuite）→ 触发执行 → 生成执行轨迹（Trace）→ 生成指标分（MetricScore）→ 报告聚合展示；过程中可推送状态事件；人工复核更新有效分。

2.6 独创架构要点

相对脚本调 API 打印结果，本软件以评测领域模型贯通全链路，以状态机约束迁移，以沙箱保证工具安全与可重复，以混合评分提升稳定性，以可视化工作台统一研发测试质检角色。该结构由李凯昕独立完成设计与实现。

第三章 功能需求与模块设计

3.1 功能结构总览

软件功能围绕“评测闭环”组织。一级功能包括：系统入口与健康检查、评测任务（Task）管理、测试用例（TestSuite）管理、评测执行引擎、Agent 执行器、工具沙箱、执行轨迹（Trace）管理、评分引擎与指标分（MetricScore）、评测报告、实时活动、安全与多租户、审计日志（AuditLog）、系统设置与前端壳层。二级功能在接口层体现为任务增删改查、用例上传、执行与取消、轨迹查询与评判、报告读取、工具列表与探测、设置读取、审计查询与 WebSocket 订阅。

输入输出规格：

输入为经鉴权的 HTTP/WebSocket 请求及环境配置；输出为 JSON 资源对象、分页列表、事件推送与持久化记录。全局约定路径前缀 `/api/v1`，错误体统一包含消息与状态码。

异常处理策略：

接入层对非法 JSON、缺参、越权与限流触发分别返回 400、401/404、429；不向客户端暴露堆栈与密钥。子系统异常不得导致进程崩溃，关键路径写结构化日志。

与其他模块交互细节：

总览模块与各业务子系统通过路由注册装配；前端壳层仅依赖公开契约，不直连数据库。健康检查汇总数据库与 Redis 等依赖状态，供部署探针使用。

3.2 评测任务（Task）管理模块

评测任务（Task）是系统聚合根。创建时写入名称、描述、Agent 配置 JSON、初始状态“已创建”、创建者 Actor。列表支持分页、状态过滤与是否包含归档。执行操作校验可执行条件并记录 Celery 任务标识以便取消。归档为软归档；删除级联测试用例（TestSuite）及其执行轨迹（Trace）与指标分（MetricScore）。租户开启时仅所有者或管理员可访问。

输入输出规格：

输入：任务名称、描述、agent_config、分页与过滤参数、任务标识、执行/取消/归档指令。输出：Task 对象（含状态、创建者、用例计数）、分页 totals、执行后状态与 celery_task_id。

异常处理策略：

名称为空拒绝创建；对终态或不存在任务的非法执行返回业务错误；取消时若异步任务已结束则以库内终态为准；越权访问统一表现为未找到，避免枚举。数据库写失败回滚并记录审计失败原因。

与其他模块交互细节：

与测试用例（TestSuite）模块一对多聚合；执行时调用评测执行引擎；状态变更经事件总线通知实时活动模块；写操作可写审计日志（AuditLog）；列表与详情经租户过滤模块约束可见性。

3.3 测试用例（TestSuite）管理模块

测试用例（TestSuite）隶属于评测任务（Task），核心字段为用户输入、期望输出、期望工具列表与扩展元数据。支持 API 批量创建与 CSV/JSON 上传。期望工具字段兼容数组、竖线与逗号分隔。缺失用户输入的行忽略，保证导入健壮性。

输入输出规格：

输入：task_id、用例数组或上传文件、可选 metadata。输出：创建条数、用例列表、单条用例详情。文件解析结果为标准化字段集合。

异常处理策略：

文件编码错误、JSON 非法、空文件返回明确错误；单行缺字段跳过并累计；超大文件受网关与应用层大小限制；无权限操作他方任务下用例时拒绝。

与其他模块交互细节：

写入后供执行引擎按任务装载；评分时以期望输出与期望工具为对照基准；删除任务时级联删除。前端详情页用例表与上传控件直接消费本模块接口。

3.4 评测执行引擎模块

执行引擎负责任务级编排：装载评测任务（Task）与全部测试用例（TestSuite）、状态迁移、逐条执行、逐条评判、失败容忍与最终汇总。子任务包括单用例执行与单轨迹评判，可并行分组。对瞬时网络错误设置重试与时间限制。Eager 与 Worker 共用业务路径。

输入输出规格：

输入：task_id、可选并行策略与超时配置。输出：更新后的任务状态、各测试用例（TestSuite）对应执行轨迹（Trace）标识、各轨迹指标分（MetricScore）落库结果、失败明细摘要。

异常处理策略：

用例为空则任务失败并提示；子任务超时写入失败信息但不必然丢弃已成功轨迹；取消请求尽力中止后续子任务；重试耗尽后标记失败；事件发布失败不影响主事务提交。

与其他模块交互细节：

调用 Agent 执行器生成执行轨迹（Trace）；调用评判引擎生成指标分（MetricScore）；通过状态机约束任务迁移；经 WebSocket 向活动模块推送；与 Celery/Redis 基础设施交互完成调度。

3.5 Agent 执行器模块

执行器抽象定义统一 run(query, tools) 结果结构，包括步骤列表、最终答案、迭代次数、Token、耗时、状态与错误信息。OpenAI 兼容实现采用 ReAct 提示与可选函数调用：每轮生成思考与动作，工具经沙箱执行并写回观察，最终答案结束循环，达最大迭代则失败。

输入输出规格：

输入：用户查询字符串、工具定义列表、模型名、最大迭代、API 端点与密钥（环境注入）。输出：steps 数组、final_answer、total_tokens、response_time_ms、status、error_message。

异常处理策略：

密钥缺失或无效时接口错误映射为失败状态；模型超时重试有限次；解析失败记录原始片段并失败退出；工具失败将错误观察写回轨迹以便评分与排障，而非静默忽略。

与其他模块交互细节：

由执行引擎按测试用例（TestSuite）调用；工具定义来自沙箱解析；输出 steps 写入执行轨迹（Trace）；Token 与耗时供报告与成本统计；不直接写指标分（MetricScore）。

3.6 工具沙箱模块

沙箱维护内置工具注册表（安全计算器、模拟检索、当前时间等）。计算器以 AST 白名单运算拒绝名称与调用节点。通用执行入线程池并设超时与输出截断。可按测试用例（TestSuite）期望工具子集解析，减少无关工具干扰。

输入输出规格：

输入：工具名、参数字典、超时秒数。输出：字符串观察结果或结构化错误信息；工具列表探测返回名称、描述与可用性。

异常处理策略：

超时返回超时错误串；非法表达式返回错误而不抛崩进程；输出超长截断；未知工具名拒绝执行；默认禁用真实外网，保证评测可重复。

与其他模块交互细节：

被 Agent 执行器在每轮 Action 时调用；探测接口供设置页与运维使用；执行结果进入执行轨迹（Trace）步骤的 observation 字段，进而影响规则工具准确率计算。

3.7 执行轨迹（Trace）与可视化模块

执行轨迹（Trace）保存用例输入快照、步骤 JSON、Token 分拆、费用估算、状态与版本元数据（Agent、提示词、模型、工具版本）。前端将步骤映射为 DAG 节点与边，并与步骤日志、评分卡片联动，使多轮推理可审查。

输入输出规格：

输入：test_suite_id、user_query 快照、steps、token 与耗时、版本字段。输出：Trace 详情、列表、前端节点边模型。查询输入含轨迹标识与租户主体。

异常处理策略：

步骤为空仍允许落库但标记失败或低分风险；JSON 过大受库与接口限制；越权读取返回未找到；版本字段缺失时允许空值但不阻断展示。

与其他模块交互细节：

由执行引擎写入；评判引擎读取 `steps` 生成指标分（`MetricScore`）；报告聚合读取；前端 `TraceFlow` 组件消费；租户模块通过任务归属过滤轨迹查询。

3.8 评分与报告模块（指标分 `MetricScore`）

评分先规则预打分，再视密钥进行 LLM 精修，并支持人工覆盖。报告按评测任务（`Task`）聚合各执行轨迹（`Trace`）的有效指标分（`MetricScore`）。每个维度保留原因说明，强调可解释而非单一黑盒总分。

输入输出规格：

输入：`trace_steps`、`expected_output`、`expected_tools`、可选人工分与审核人。输出：各维分数、总分、`reason`、`mode`（`rule_only/hybrid`）、持久化 `MetricScore` 记录与报告聚合结构。

异常处理策略：

LLM 超时或 JSON 解析失败降级为规则分；无期望工具时工具维满分；人工复核缺审核人时可拒绝或记默认；重复评判以最新业务策略更新并保留可追溯字段。

与其他模块交互细节：

依赖执行轨迹（`Trace`）与测试用例（`TestSuite`）期望；结果供报告与前端评分卡片；人工复核写回同一指标分（`MetricScore`）实体的人评字段；缓存键与步骤内容哈希相关以降低重复成本。

3.9 安全、租户、审计与实时活动模块

鉴权支持 X-API-Key 与 Bearer，常量时间比较。租户以 `Actor` 作为评测任务（`Task`）所有者标记并过滤列表与执行轨迹（`Trace`）。审计日志（`AuditLog`）记录关键写操作。`WebSocket` 推送任务状态。限流保护公共接口。

输入输出规格：

输入：请求头密钥、Actor、审计动作与对象、WebSocket 订阅参数。输出：AuthIdentity、过滤后的查询、AuditLog 记录、状态事件消息。

异常处理策略：

密钥错误返回 401；越权返回未找到；审计写失败记录日志但不回滚主业务（或按配置严格模式）；推送失败尽力而为；限流触发 429。

与其他模块交互细节：

横切嵌入路由依赖与中间件；任务与轨迹 API 调用租户过滤；执行引擎发布事件；设置页展示鉴权提示；与管理员 Actor 列表配置联动。

第四章 数据库设计思路

4.1 设计原则

以评测业务为中心，聚合清晰、级联明确、可追溯、可扩展。主键字符串化便于传递；JSON 承载配置与步骤；枚举安全存储；时间戳完备。

4.2 E-R 关系文字描述

核心实体：Actor（逻辑）、评测任务（Task）、测试用例（TestSuite）、执行轨迹（Trace）、指标分（MetricScore）、审计日志（AuditLog）。关系：Actor 1—n Task；Task 1—n TestSuite；TestSuite 1—n Trace；Trace 1—n MetricScore。删除任务级联删除用例子树，避免孤儿数据。

4.3 主要实体字段语义

Task：名称、描述、状态、agent_config、celery_task_id、归档、创建者。TestSuite：task_id、user_query、expected_output、expected_tools、metadata。Trace：test_suite_id、输入快照、steps、token、耗时、状态、版本元数据。MetricScore：

trace_id、指标名、分数、原因、置信度、人评字段。AuditLog：动作、对象、主体、请求标识、时间。

4.4 状态字段与约束思想

应用层强制任务状态机；轨迹区分成功失败；指标分机评与人评并存，有效分优先人评。SQLite 演示可缺列回填；生产以 Alembic 为准。

4.5 索引与查询路径

任务分页、按任务过滤用例、按用例过滤轨迹、按轨迹读分数、按所有者连接过滤、审计时间倒序。报告以任务为入口聚合，保证权限边界一致。

4.6 数据一致性与幂等

先写执行轨迹（Trace）再写指标分（MetricScore）；评分失败可重评；取消尽力中止；归档软删除满足审计。

第五章 核心算法与处理逻辑

5.1 任务状态机算法

状态机定义从创建出发，经排队、运行、等待工具、评判到达完成的主路径，以及失败、取消、超时等异常终态。每个状态仅允许迁移到白名单集合，终态不可再迁。该算法把分布式异步过程中的阶段变成可判断、可展示、可审计的显式知识。

算法伪代码（文字版）：

步骤 A：读取评测任务（Task）当前状态 S 与目标状态 T。

步骤 B：若 S 属于终态集合，则拒绝迁移并返回非法迁移错误。

步骤 C：查询 allowed_transitions(S)，若 T 不在集合中则拒绝。

步骤 D：在同一事务内将状态更新为 T，写入更新时间，必要时写入 celery_task_id。

步骤 E：尽力发布状态变更事件（含 prev=S、curr=T、actor）。

步骤 F：提交事务并返回最新任务对象。

边界条件处理：

空任务标识、并发双写、取消与完成竞态、从失败再次执行（若业务允许则需显式重置策略，当前实现以状态机为准拒绝非法回流）。终态集合包含完成、失败、取消、超时。

性能复杂度分析：

单次迁移为 $O(1)$ 字典查询与一次行级更新；事件发布为 $O(1)$ 消息写入。相对评测主耗时可忽略。瓶颈不在状态机而在模型调用。

5.2 ReAct 多轮执行逻辑

执行器在最大迭代次数内循环，请求大模型，解析思考、动作与动作输入；工具则沙箱执行并追加观察；最终答案则成功结束；达上限则失败。

算法伪代码（文字版）：

步骤 A：初始化 messages=[system, user_query]，steps 空表，tokens=0，iter=0。

步骤 B：若 iter 达到 max_iterations，返回 status=max_iterations_reached 与已有 steps。

步骤 C：调用聊天补全接口，累计 token，得到 content 或 tool_calls。

步骤 D：解析 action 与 action_input；构造 ReActStep 写入 steps。

步骤 E：若 action 为 final_answer，返回 success 与 final_answer。

步骤 F：若 action 为工具名，调用沙箱得到 observation，将观察追加 messages，iter 自增，回到 B。

步骤 G：若解析失败或接口异常，返回 failed 与 error_message。

边界条件处理：

tools 为空时仅允许直接最终答案；模型返回空内容记失败；工具超时写入错误观察继续或结束视策略；温度固定偏低以提升评测稳定性。

性能复杂度分析：

时间复杂度主要是 $O(K)$ 次外部模型调用，K 为 max_iterations，单次调用延迟由网络与模型决定；空间复杂度 $O(K)$ 保存 steps 与消息历史。K 通常为个位数，属可接受评测开销。

5.3 工具安全执行逻辑

对计算器：限长、AST 白名单、禁止 Name/Call/Attribute。对通用工具：线程超时、输出截断、未知工具拒绝。

算法伪代码（文字版）：

步骤 A：校验工具名存在于注册表。

步骤 B：规范化参数字典。

步骤 C：提交线程池执行，等待 timeout。

步骤 D：若超时，返回超时错误串。

步骤 E：若异常，返回失败错误串。

步骤 F：将结果转字符串，超 MAX_OUTPUT_CHARS 则截断并标注。

步骤 G：返回观察字符串。

边界条件处理：

空表达式、超长表达式、非数值节点、除零、嵌套过深（由解析失败覆盖）、并发多工具受线程池 worker 数限制。

性能复杂度分析：

AST 遍历相对表达式长度 n 为 $O(n)$ ；算术求值 $O(n)$ ；超时上界为常数 T 秒。整体相对模型调用可忽略。

5.4 规则预评分算法

三维评分：工具准确率、答案正确性、推理连贯性，加权合成预总分并生成原因。

算法伪代码（文字版）：

步骤 A：从执行轨迹（Trace）steps 提取实际工具名集合 A ，读取期望工具集合 E 。

步骤 B：若 E 为空，工具百分制=100；否则对 missing 与 extra 各扣固定分，下限 0。

步骤 C：工具维得分 = 百分制 * 0.4。

步骤 D：抽取最终答案文本 Ans ，与期望输出 Exp 做词项重叠或归一化启发，映射到答案维 0-40。

步骤 E：按步骤数、是否存在最终答案、是否空动作等启发计算连贯维 0-20。

步骤 F：总分 = 三维之和；拼接 reason；mode=rule_only。

边界条件处理：

无期望输出时答案维基线满分或中性分；无答案可抽取时答案维 0；steps 为空时各维低分；工具名大小写与别名需在提取阶段规范化。

性能复杂度分析：

集合差为 $O(|A|+|E|)$ ；文本分词重叠近似 $O(L)$ ；相对 LLM 评判可忽略，适合批量预打分与无密钥场景。

5.5 LLM 精修与降级策略

有密钥时组织提示要求仅输出 JSON 分数；成功则 hybrid；失败回退规则分。

算法伪代码（文字版）：

步骤 A：计算规则预分 pre。

步骤 B：若无有效密钥，返回 pre。

步骤 C：构造 system/user 提示，包含 steps 摘要、期望与 pre。

步骤 D：请求模型 JSON；解析 scores/total/reason。

步骤 E：解析成功则写 mode=hybrid 并返回；失败或异常则记录警告并返回 pre。

步骤 F：可选将结果写入内容哈希缓存，命中则直接返回。

边界条件处理：

密钥占位符 `your_api_key_here` 视为无效；缓存超过容量清空；模型返回非 JSON 或缺维时用 pre 补齐；重试采用有限次数指数退避。

性能复杂度分析：

额外 1 次模型调用主导耗时；缓存命中 $O(1)$ 。批量评测时缓存可显著降本。

5.6 人工复核有效分算法

若 `is_human_reviewed` 且 `human_score` 非空，有效分=`human_score`，否则=机评分。

算法伪代码（文字版）： 读取指标分（MetricScore）记录；判断人评标记；返回有效分；报告聚合使用有效分。

边界条件处理： 仅标记未填分、仅填分未标记、分数越界（应在写入时校验 0-100）。

性能复杂度分析： $O(1)$ 字段读取。

5.7 异步编排与聚合

算法伪代码（文字版）：

步骤 A：装载评测任务（Task）与测试用例（TestSuite）列表。

步骤 B：状态→运行并广播。

步骤 C：对每个 suite 调用执行子任务，落库执行轨迹（Trace）。

步骤 D：对每个 trace 调用评判子任务，落库指标分（MetricScore）。

步骤 E：统计失败数，状态→完成或失败并广播。

边界条件处理：部分失败仍保留成功轨迹；取消打断后续循环；子任务重试不重复污染时需幂等设计（以新轨迹或更新策略为准）。

性能复杂度分析：设 N 个用例，串行为 $O(N)$ 次执行加 $O(N)$ 次评判；分组并行为近似 $O(N/P)$ 墙钟时间， P 为并发度，受 Worker 与模型限流约束。

5.8 成本与版本元数据

轨迹记录 prompt/completion tokens、费用估算与版本号，支撑“分数产生条件”解释。

算法伪代码（文字版）：执行结束写入 token 字段；按单价估算 cost；写入 agent/prompt/model/tool version；报告展示只读。

边界条件处理：缺 usage 时 token 记 0；版本未知可空。

性能复杂度分析： $O(1)$ 附加字段写入。

第六章 模块间数据流转

创建流：前端提交 → Task 落库 → 上传解析 → TestSuite 批量插入。执行流：执行指令 → 状态迁移 → 执行器 → Trace → 事件推送。评分流：Trace+期望 → 评判 →

MetricScore → 报告。查询流：租户过滤贯穿列表与详情。前端状态流：服务端真相 + 本地交互缓冲 + DAG/评分卡片渲染。

第七章 接口规范设计

REST+JSON，前缀 `/api/v1`。任务、用例、执行、轨迹、评判、报告、工具、设置、审计、健康检查与 WebSocket 活动。鉴权开启时除公共路径外需 API Key。分页设上限。执行与评分注意重复提交与竞态下的终态一致性。

第八章 前端工作台设计

信息架构：总览、任务、报告、设置；详情页承载用例、执行、执行轨迹（Trace）DAG、指标分（MetricScore）。可视化原则：节点表达步骤，日志表达原文，卡片表达分数，三者联动。契约与后端字段对齐，保证材料术语一致。

第九章 安全设计

认证授权、输入与工具安全、密钥仅环境配置、限流与请求 ID、审计追责。占位符：`your_api_key_here`；禁止材料出现真实密钥与内网 IP。

第十章 可靠性、性能与可扩展性

任务超时与重试、失败隔离、事务边界、单元与端到端测试；异步扩展 Worker；评分缓存；执行器/工具/指标可扩展。

第十一章 运行部署设计

本地 Eager 与 Docker 全栈双路径；配置分层；生产开启鉴权租户、关闭不必要文档暴露。

第十二章 测试与质量保障

覆盖鉴权、租户、沙箱、执行器、评分、任务 API；验收要点：创建任务、导入测试用例（TestSuite）、执行产生执行轨迹（Trace）、规则分与 hybrid、人工复核、隔离、报告可读。全流程测试验证由著作权人李凯昕独立完成。

第十三章 独创性综合论述

13.1 相对通用框架的差异

本软件不是对 FastAPI 或 React 的简单演示，也不是单一聊天页面。独创性体现在：评测领域驱动的数据模型与状态机；Agent 执行与工具沙箱一体化的执行轨迹（Trace）体系；规则与 LLM 与人工复核的三级评分闭环及指标分（MetricScore）可解释结构；异步编排与 Eager 共用业务路径；面向评测人员的 DAG 工作台与企业级横切能力组合。

13.2 权利人独立完成声明

系统架构设计、核心算法编码、领域模型（评测任务 Task / 测试用例 TestSuite / 执行轨迹 Trace / 指标分 MetricScore）定义，以及全流程测试验证，均由著作权人李凯昕独立完成。所使用的开源组件遵循其许可证；大模型服务为外部依赖；本软件主张权利的对象为原创程序与文档表达。

13.3 原创性边界声明

不主张第三方库与模型权重权利；源程序鉴别材料选取核心业务文件，排除第三方库与构建产物。

13.4 架构优化贡献说明

李凯昕完成：评测过程阶段显式状态化；工具沙箱隔离；可降级混合评分；Actor 轻量多租户；前后端契约与模块清单同源维护。上述构成技术特点与原创性支撑。

第十四章 总结

AgentFlow-Eval Agent 自动化评测工作台以评测任务（Task）与测试用例（TestSuite）为入口，以异步或同步方式执行 ReAct Agent，以执行轨迹（Trace）支撑可视化与评分，以规则、大模型与人工复核形成可信指标分（MetricScore），并以鉴权、租户、审计与实时活动满足基本治理需要。本说明书与申请表、用户手册、源程序核心模块相互印证，达到登记对设计说明深度与一致性要求。

第十五章 评测场景建模补充说明

在企业实际落地过程中，评测场景往往呈现多样性。客服类场景关注是否正确查询订单与知识库，是否在未知时拒答；办公助手场景关注是否正确调用日历与邮件类工具；数据分析场景关注计算工具使用是否正确以及最终数值是否一致。本软件通过“用户输入、期望输出、期望工具”三元组对场景进行统一抽象，使不同业务线可以共用同一套执行与评分基础设施。该抽象由李凯昕在需求分析阶段提炼，避免为每个业务定制互不兼容的评测脚本。统一抽象带来的直接好处是测试用例（TestSuite）可迁移、指标可复用、报告可对照。对于复杂场景，可通过扩展元数据承载业务标签，例如难度、业务线与优先级，而不破坏主干模型。场景建模还要求同一评测任务（Task）内用例风格一致，否则模型行为方差会放大分数噪声；因此手册建议按业务线拆分任务。场景标签可在报告侧用于分组统计，为后续版本扩展预留，而不阻塞 V1.0 闭环。

第十六章 提示词与模型配置管理思想

Agent 与评判器的行为强烈依赖模型名称、温度、最大轮次与系统提示。本软件将任务级 Agent 配置以 JSON 持久化于评测任务（Task），使同一任务在不同时间重复执行时具备配置快照意义。执行轨迹（Trace）版本元数据进一步把实现版本与模型版本写入记录，形成“配置加版本加轨迹”的三维追溯。设计上不把提示词硬编码为不可替换常量，而是在执行器内集中管理 ReAct 模板，便于后续抽取为可版本化资源。李凯昕在此强调：评测系统必须能回答“当时用的什么模型与提示”，否则指标分（MetricScore）无法用于回归比较。配置变更应通过新任务或明确重跑产生新轨迹，而不是静默覆盖历史解释条件。密钥与端点仅环境注入，不进入配置 JSON 明文提交材料。

第十七章 失败模式与恢复策略

评测过程常见失败包括：模型服务超时、密钥无效、工具执行超时、测试用例（TestSuite）为空、任务被取消、数据库不可写等。系统设计上将失败分类处理。对瞬时网络错误，子任务可重试；对业务性失败，写入失败执行轨迹（Trace）或失败任务状态并给出错误信息；对用户取消，尽快停止后续用例并落终态。评分失败保留轨迹以便重评。前端对失败状态使用明确徽章与提示，避免静默失败。恢复策略上，支持在修复配置后重新执行评测任务（Task）或对单条执行轨迹（Trace）重新评判，而不必重建全部用例。该策略降低了真实业务中的运维成本，并与状态机终态设计一致。

第十八章 多租户隔离的威胁模型

在 API Key 多使用者场景下，主要威胁是横向越权读取他人评测任务（Task）、测试用例（TestSuite）与执行轨迹（Trace）中的业务敏感问题。本软件威胁模型假设密钥可能被多人持有，因此在列表、详情、轨迹与执行等路径统一实施所有者

校验，管理员名单用于运维旁路。返回未找到而非明确拒绝，减少资源枚举。

WebSocket 推送也应只面向相关主体活动，避免广播泄露。审计日志（AuditLog）用于事后发现异常访问模式。该轻量模型不替代完整组织级权限系统，但在中小团队评测平台中达到成本与安全的合理平衡，是李凯昕针对产品阶段做出的务实安全设计。

第十九章 指标解释性与报告阅读方法

自动评分若不可解释，则难以指导改进。因此每个指标分（MetricScore）维度保留原因字段，规则层写明缺失或多余工具，模型层写明扣分理由，人工层记录审核者。报告阅读建议先看评测任务（Task）总分分布，再下钻低分测试用例（TestSuite），结合执行轨迹（Trace）DAG 定位出错步骤，最后决定是改提示、改工具描述还是改业务预期。软件通过数据结构支持这一阅读方法，而不是只给一个黑盒分数。解释性设计使评测结果能够进入研发迭代闭环，体现工作台产品价值。

第二十章 与持续集成的协同

虽然本软件是交互式工作台，但其 API 化设计允许被持续集成系统调用：导入测试用例（TestSuite）、触发执行、拉取报告。仓库内单元测试与端到端测试保障核心逻辑稳定。未来可在外部流水线中把关键 Agent 场景作为门禁。设计上保持接口稳定与健康检查友好，是为这种协同预留的接口。李凯昕在总体设计中将“人机交互评测”与“自动化回归”视为同一数据模型上的两种消费方式，从而避免两套体系。

第二十一章 前端性能与可访问性考虑

任务详情页信息密度高，设计上采用分区展示：基本信息、测试用例（TestSuite）表、执行控制、执行轨迹（Trace）列表、DAG、日志与指标分（MetricScore）。大数据量步骤时，日志区应可滚动，图节点应避免一次性渲染失控。路由级拆分减少初始包体。状态徽章与文字标签并用，降低仅靠颜色传达信息的问题。空状态与错误边界提升稳健性。这些界面工程细节服务于长时间评测操作，属于产品化独创表达的一部分。

第二十二章 数据留存与合规建议

执行轨迹（Trace）中可能包含业务侧用户问题与中间推理，属于敏感数据。建议在企业内部部署时限制访问密钥分发，开启租户隔离与审计，并按组织要求设置留存周期。本软件提供删除与归档能力作为治理手段。日志应避免记录完整密钥与不必要的个人信息。登记材料截图需脱敏。合规并不是单独模块，而是贯穿鉴权、审计、脱敏与部署建议的设计约束。

第二十三章 扩展点清单

执行器扩展点：实现基类并在工厂注册。工具扩展点：在沙箱注册新的纯函数工具并暴露描述。指标扩展点：在规则层增加计算函数并在报告展示指标分（MetricScore）。存储扩展点：通过配置切换数据库后端。通知扩展点：在事件发布后增加企业 IM 适配。前端扩展点：新增页面挂路由。清晰扩展点使软件在保持核心稳定的同时可持续演进。

第二十四章 设计约束与非目标

本软件非目标包括：不内置训练或微调大模型；不作为通用 MLOps 平台管理所有模型资产；不在沙箱内提供无限制外网浏览；不替代专业渗透测试与企业 IAM。明确非目标有助于保持产品边界清晰，把复杂度投入到 Agent 评测闭环这一核心价值。李凯昕在需求裁剪上坚持“闭环完整优先于功能堆叠”，这是本软件能够在可运行形态下完成鉴定与演示的重要原因。

第二十五章 关键算法伪流程文字版（总流程）

全流程可描述为：输入为评测任务（Task）标识；输出为更新后的任务状态与持久化执行轨迹（Trace）及指标分（MetricScore）。步骤一读取任务与测试用例（TestSuite）集合；步骤二若集合为空则失败返回；步骤三状态改为运行并广播；步骤四对每一用例调用执行器得到步骤数组并落库为执行轨迹（Trace）；步骤五对每一轨迹调用评判器得到指标分（MetricScore）并落库；步骤六汇总失败计数；步骤七状态改为完成或失败并广播；步骤八结束。执行器子流程与评判子流程详见第五章。上述文字流程与实现保持一致，便于评审理解。

第二十六章 模块职责矩阵说明

路由模块只做协议适配；领域模型只做数据与状态表达；执行器只做多轮推理；沙箱只做工具安全执行；评判器只做指标分（MetricScore）生产；编排器只做流程粘合；前端只做人机交互。禁止在路由中写复杂评分，禁止在前端计算权威分数，禁止在执行器中直接写报告聚合。职责矩阵减少耦合，使单元测试可以针对纯函数与类方法进行。该工程约束由总体设计明确规定，并在代码目录上落实。

第二十七章 界面—接口—表结构一致性

软著材料常见问题是说明书、界面与代码三套说法不一致。本项目以功能模块清单为中枢：界面菜单对应路由，路由对应接口，接口对应表字段。例如任务列表页对应任务列表接口对应 `tasks` 表（评测任务 Task）；DAG 对应轨迹详情接口对应 `traces.steps` 字段（执行轨迹 Trace）；评分卡片对应 `metric_scores` 表（指标分 MetricScore）；用例表对应 `test_suites` 表（测试用例 TestSuite）。本设计说明书与用户手册沿用同一术语，确保三套材料完全一致。

第二十八章 后续演进路线（不作为当前版本范围）

可能的演进包括更细的角色权限、更多连接器工具、批量对比实验、评分校准集、团队空间与更丰富的报表导出。无论怎样演进，当前 **V1.0** 已经形成最小完整闭环，满足独立软件的功能完整性。演进路线不削弱现有原创性，只说明系统具备生长空间。当前登记版本以 **V1.0** 可运行实现为准，开发完成日期为 2026 年 7 月 14 日。

第二十九章 设计说明书结论重申

综合前述章节，AgentFlow-Eval Agent 自动化评测工作台在需求定义、架构分层、数据模型、核心算法、接口与安全、前端交互等方面均具备完整设计与实现对应关系。**系统架构设计、核心算法编码、领域模型定义及全流程测试验证，均由著作权人李凯昕独立完成。** 本说明书纯文字阐述满足登记实践对设计类文档深度的要求，并与申请表功能特点、用户手册及源程序核心模块相互印证，特此作为文档鉴别材料提交。

文档结束。